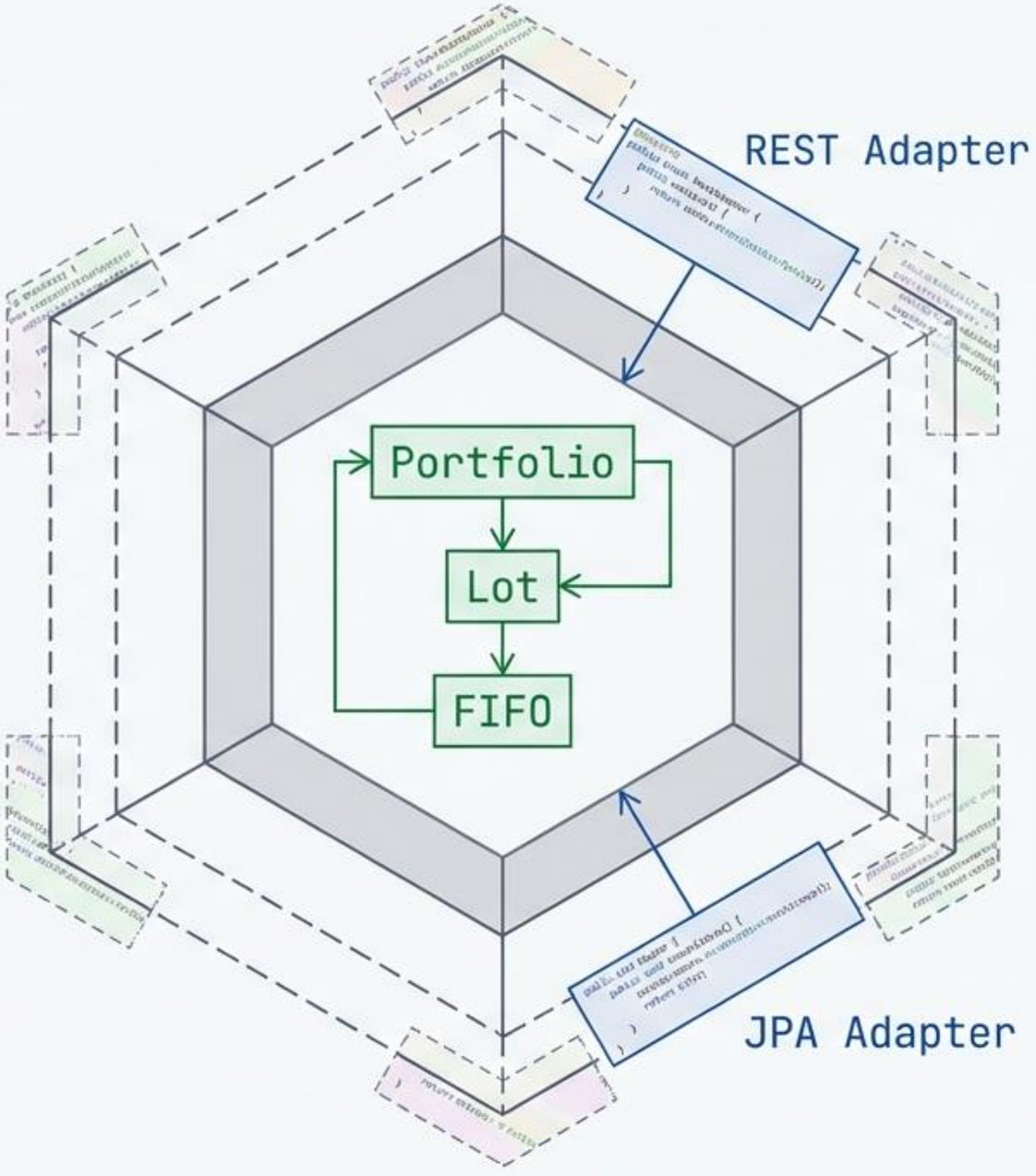
Domain-Driven Design & Hexagonal Architecture in Action

A Production-Grade Approach with HexaStock

- Protecting boundaries.
- Enforcing invariants.
- Proving architecture.
- Proving architecture through code.

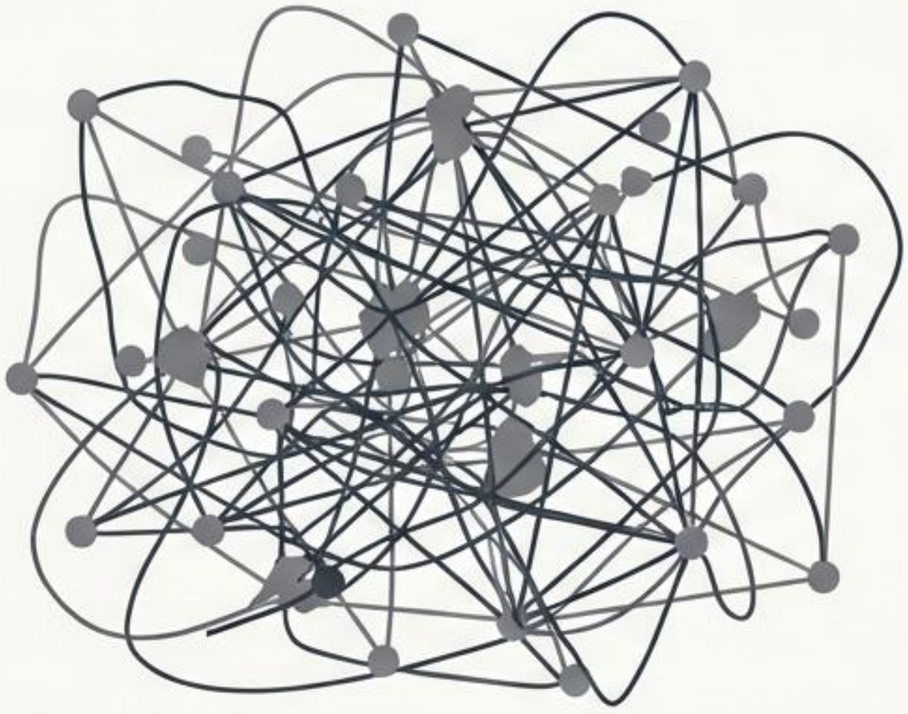


NotebookLM

Architecture in the Age of AI

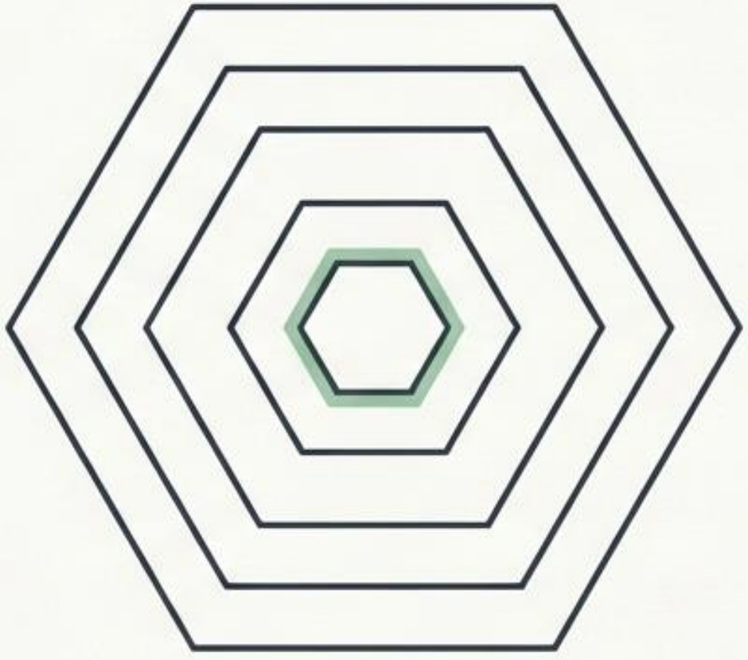
Alfredo Rueda Unsain | Software Architect & Professor

The Reality



AI accelerates implementation, but AI cannot define your `aggregate boundaries` or negotiate a `Ubiquitous Language`. Architecture remains a profoundly human responsibility.

The Objective



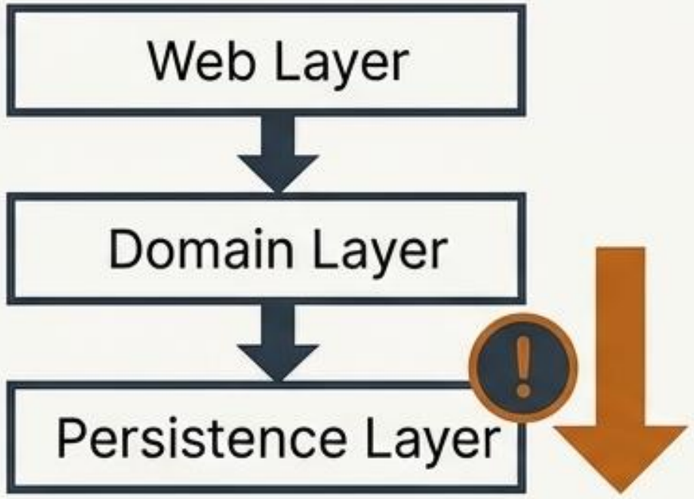
Build systems where:

1. Business rules never drift.
2. Complexity is contained.
3. Architecture is verified by `CI`.

NotebookLM

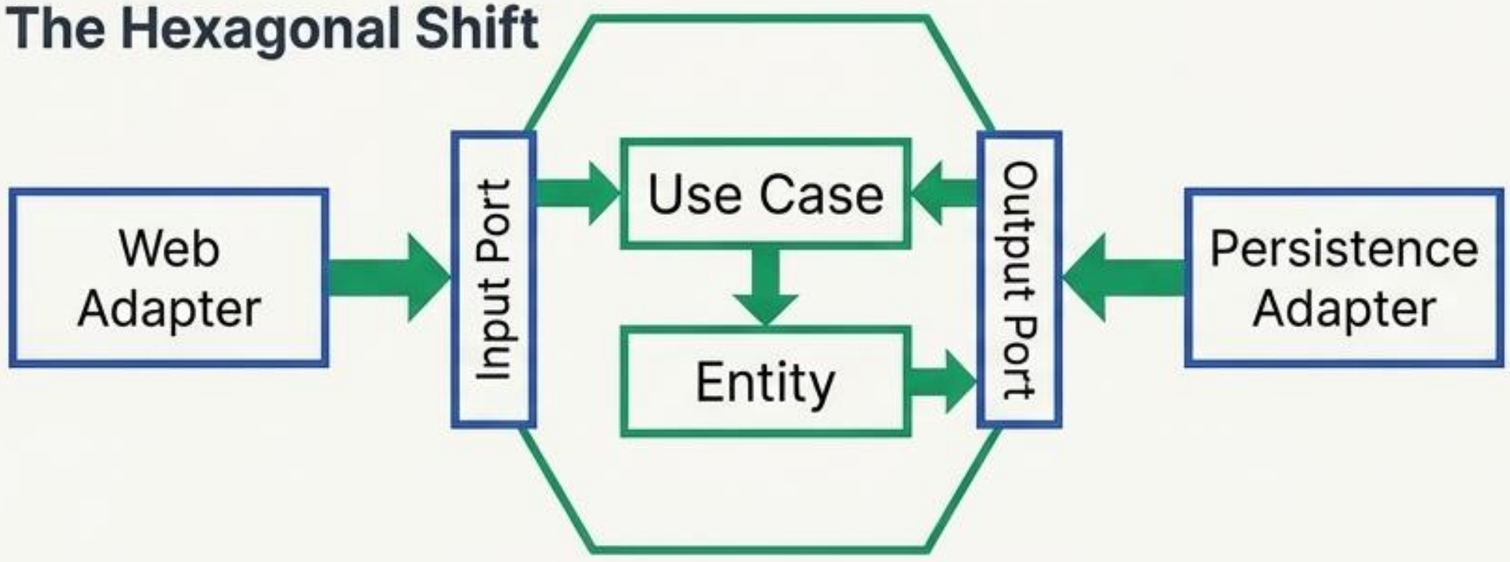
The Flaw of Traditional Layers

The Layered Trap



The Consequence: Business logic leaks into controllers, UI dictates entities, and testing requires a full database.

The Hexagonal Shift



Invert the dependencies.
The core depends on nothing.
Adapters plug INTO the core.

NotebookLM

Structure Meets Meaning

Hexagonal Architecture

The Structural Vault.

Provides the organization.

Isolates business logic from external infrastructure via Ports and Adapters.

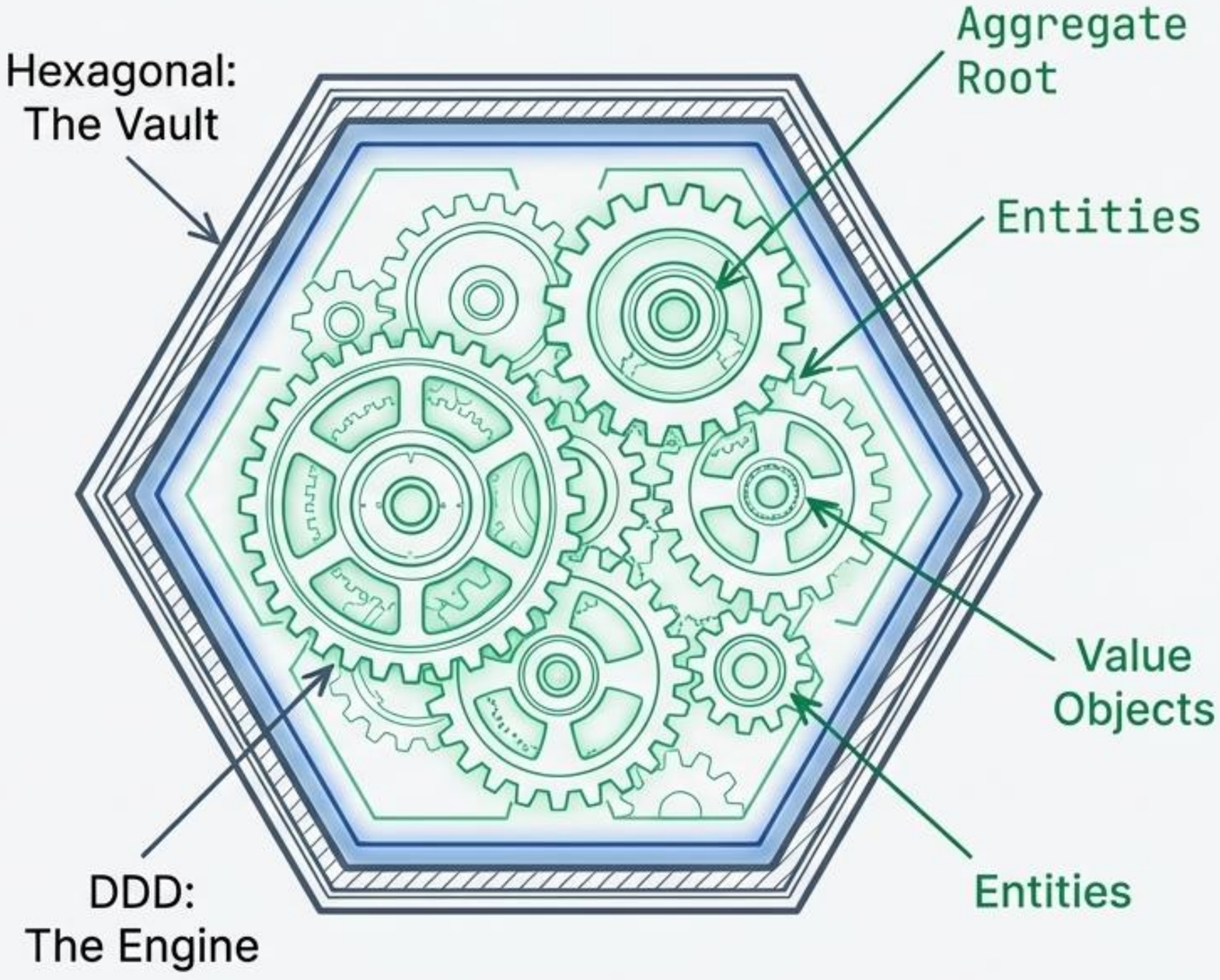
Domain-Driven Design (DDD)

The Engine

Provides the modeling methodology.

Shapes the internal core using **Aggregates**, **Entities**, **Value Objects**, and **Ubiquitous Language**.

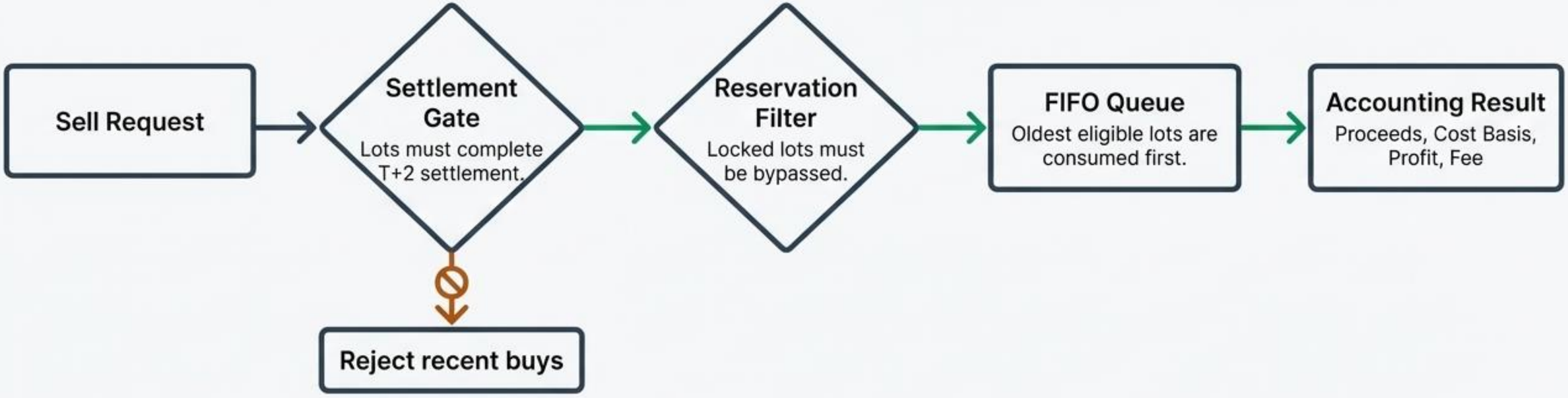
The Synergy: Hexagonal Architecture does not require DDD, but DDD makes the protected core expressive and valuable.



NotebookLM

Deep Dive: The Sell Stocks Use Case

Selling shares is not a simple CRUD update.

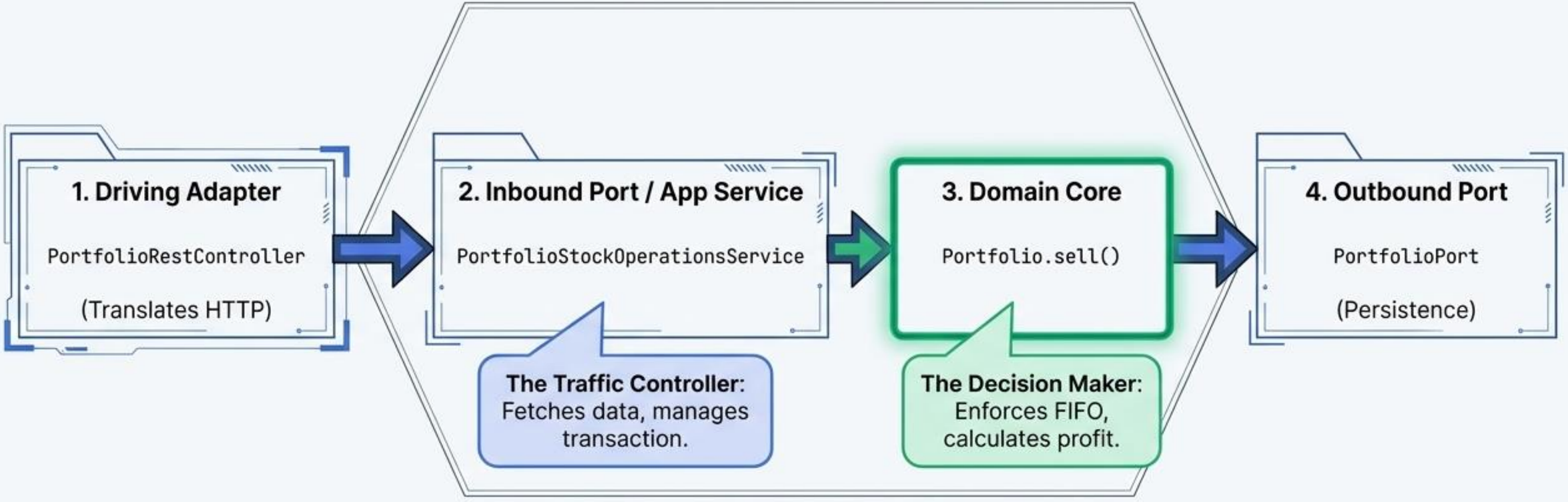


Financial Accounting Identity Must Hold Atomically:

$$\text{Cost Basis} + \text{Profit} + \text{Fee} = \text{Proceeds}$$

NotebookLM

Orchestration vs. Execution

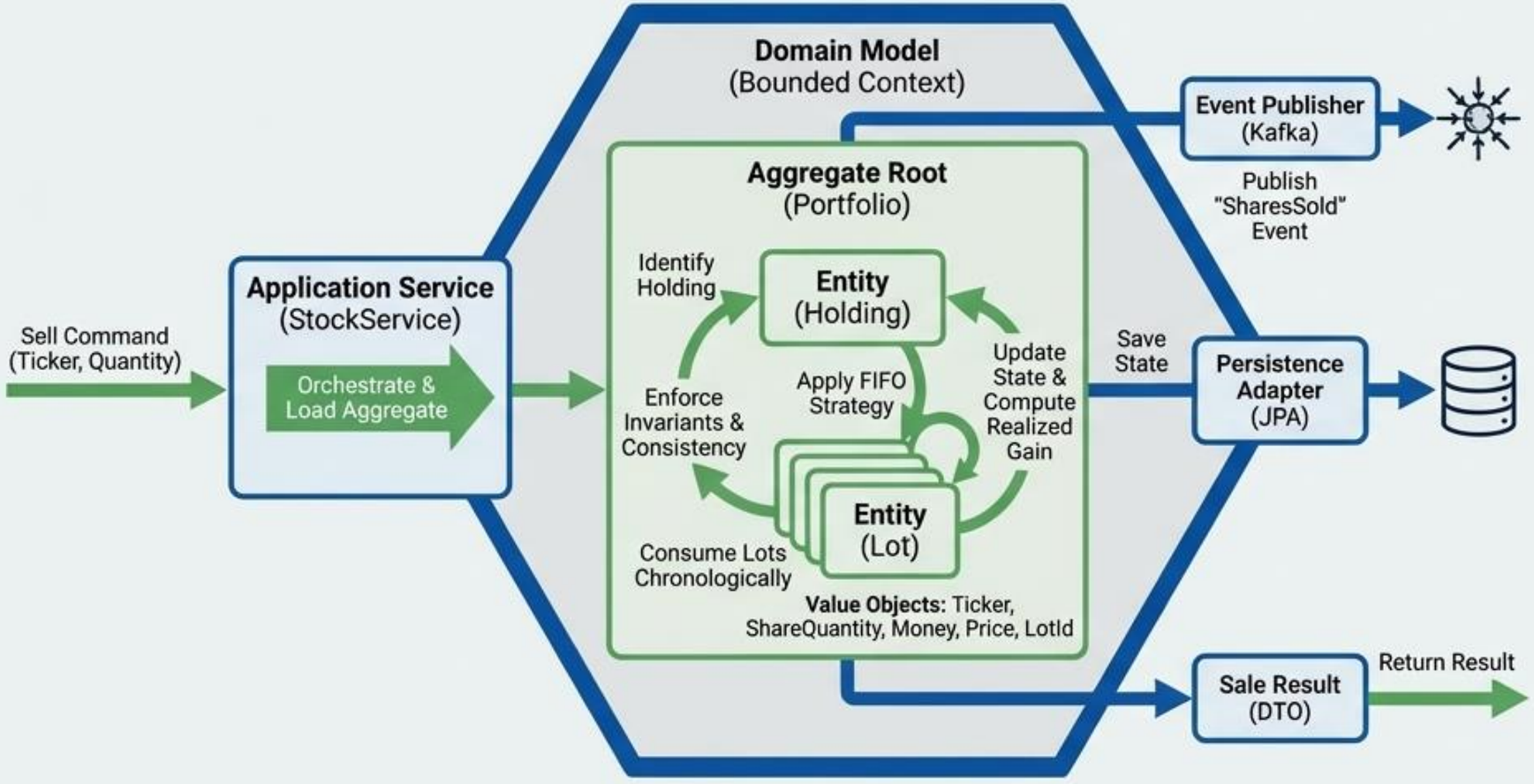


Notice the strict separation of roles. The Application Service orchestrates, but it makes zero business decisions. It delegates entirely to the Portfolio aggregate, which guarantees state transitions.

NotebookLM

Slide A: Deep Dive: Implementing “Sell Stocks” with Domain-Driven Design & Hexagonal Architecture

A Production-Grade Walkthrough with HexaStock

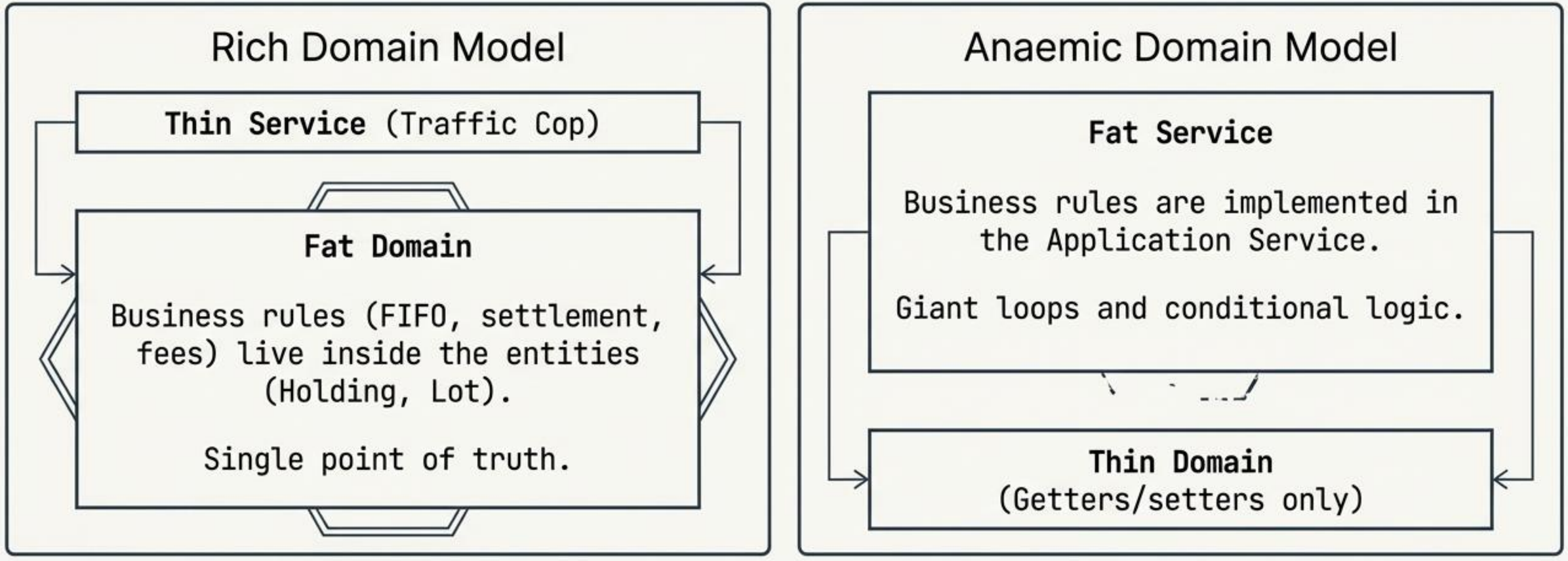


- Unpacking the Bounded Context & Ubiquitous Language.
- Mastering the Aggregate Root as a Consistency Boundary.
- Implementing Complex Business Logic (FIFO) within the Domain.
- Leveraging Value Objects for Expressive & Safer Code.
- Protecting Invariants in a Concurrent Environment.
- Integrating Domain Logic with Hexagonal Architecture.

NotebookLM

The Decisive Battle: Rich vs Anaemic Domain

Both models pass day-one tests. What happens when the system evolves?



The most consequential decision you will make is where business logic lives.
Let's look at HexaStock's A/B test of these two approaches.

NotebookLM

Rule Drift and Architectural Failure

The Flaw: In the anaemic model, a new endpoint calls `Lot.isAvailableForSale()` directly, bypassing the service logic. The domain method is stale and forgets to check `!reserved`.

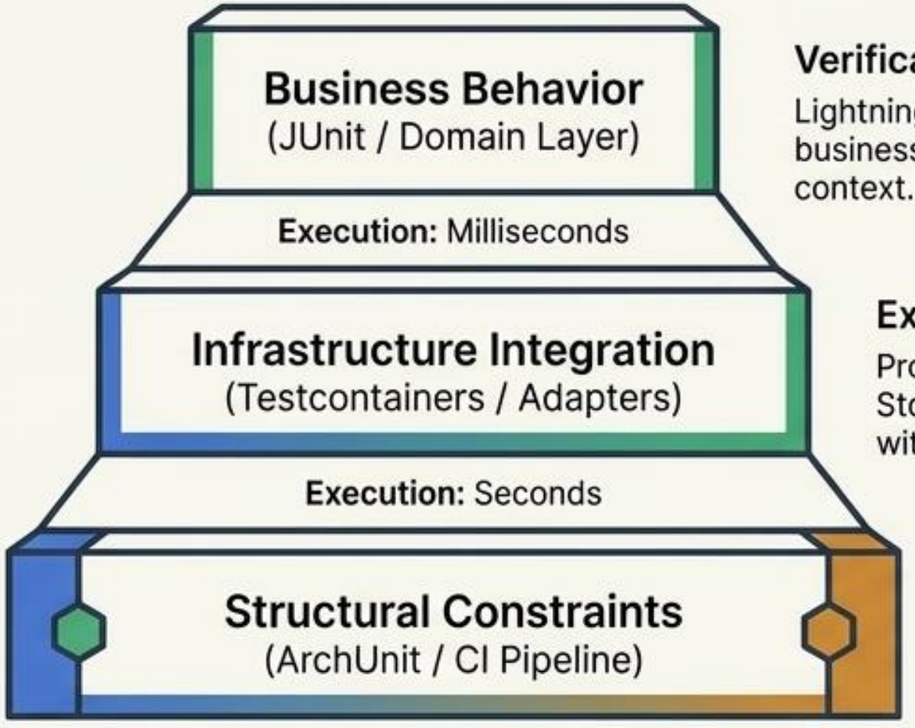
	Metric	Rich Branch	Anaemic Branch
1	Rule Location	`Lot Entity`	Scattered in Services
2	Bug Surface	Zero duplication	Stale domain methods
3	Test Outcome (Same 170 tests)	✅ 170 / 170 Pass	❌ 10 Failures. Fails `ATOMIC-01` scenario. Reserved lots illegally sold.

Because the entity owns its truth in a rich model, this class of bug becomes impossible.

NotebookLM

Protecting the Architecture

Enforcing boundaries through testing and CI.



Verification Table

Lightning-fast execution of business invariants. No Spring context. No databases.

Execution Time

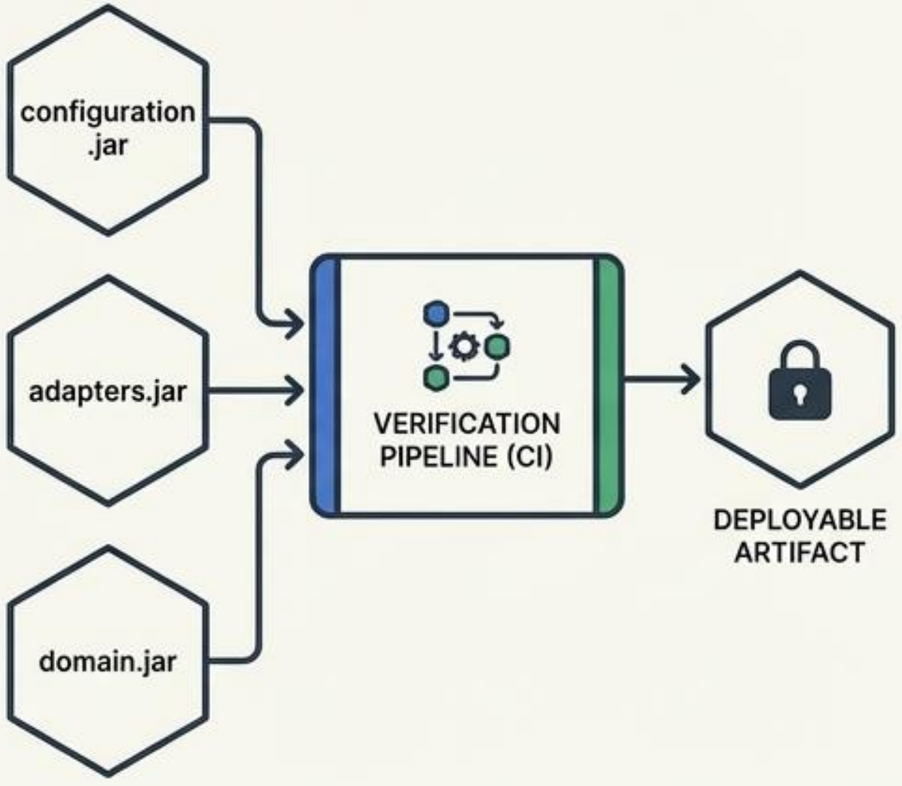
Proving we can swap the StockPriceAdapter seamlessly without changing the domain.

Failure and Integrity

A build **FAILS** if the precisation annotraint.

Result: Architectural integrity locked

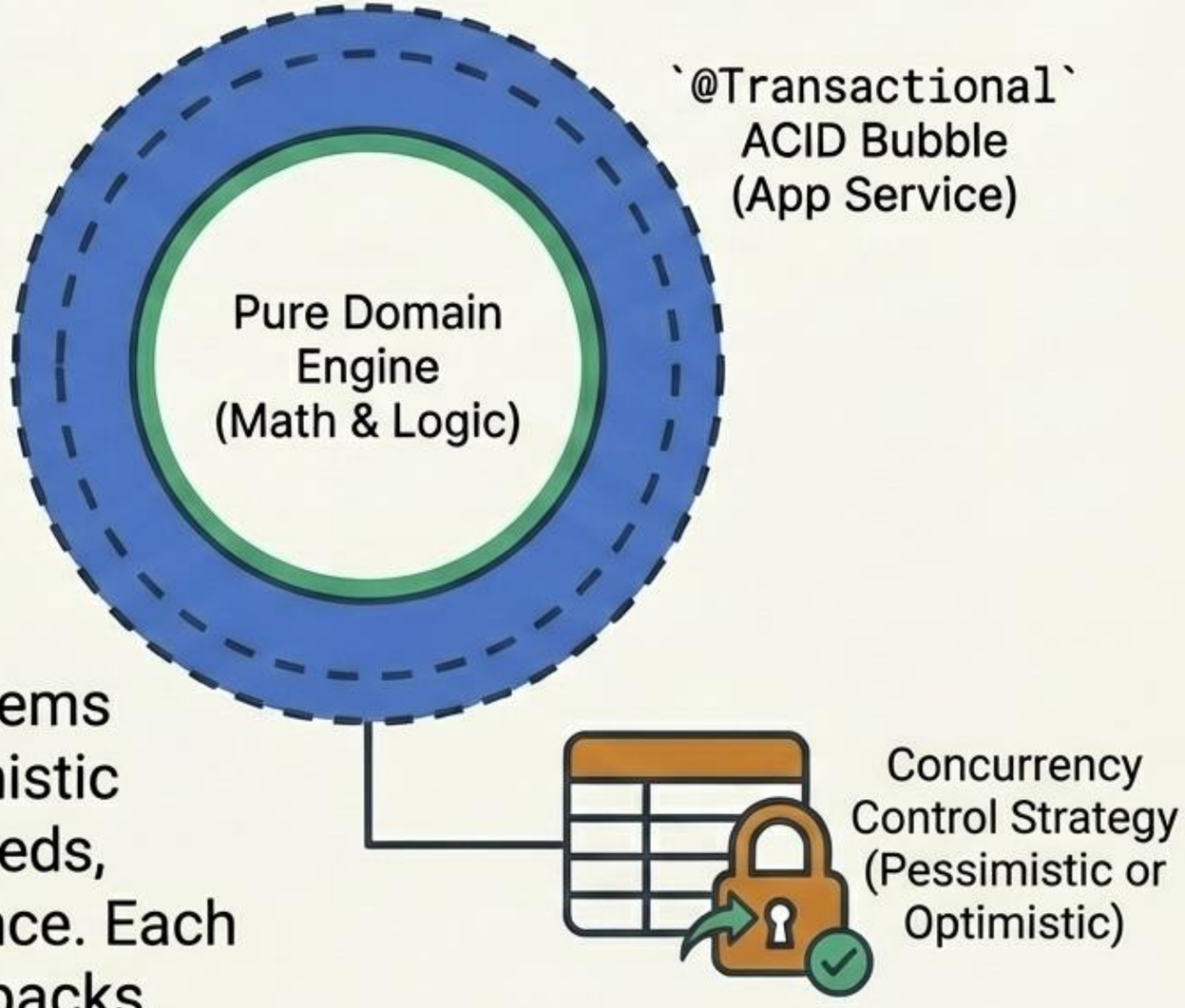
A build FAILS if the Domain accidentally imports a Spring annotation or a Web class.



NotebookLM

Purity Meets Production Realities

- **The Reality:** Pure domains don't handle database commits or race conditions.
- **The Solution:** Application Services apply `@Transactional` boundaries.
- **Concurrency at Scale:** Real-world stock selling requires explicit concurrency control. Production systems may use pessimistic locking or optimistic locking depending on consistency needs, contention patterns, and retry tolerance. Each approach has advantages and drawbacks.

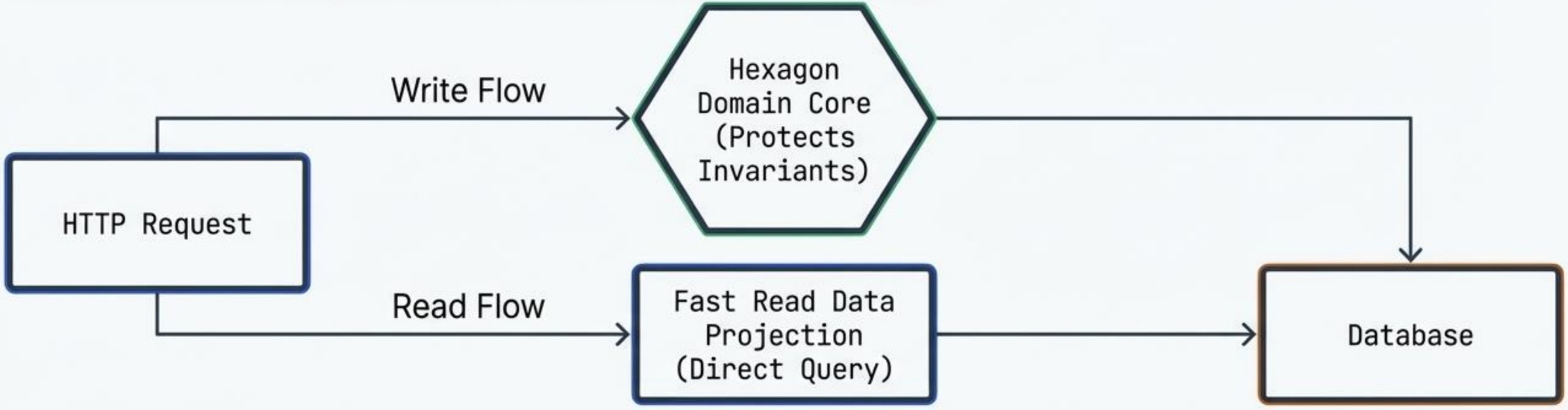


NotebookLM

Evolving the Architecture: CQRS

Market Sentinel Watchlists

The Problem: Rich domain models protect invariants (writes) but are inefficient for complex, read-heavy dashboards.



The Evolution: Command Query Responsibility Segregation (CQRS). Bypass the domain entirely for reads. Use optimized read-models and direct queries, preserving the Aggregate strictly for writes.

NotebookLM

Specification-First Engineering

Ubiquitous Language ensures the conceptual chain is never broken.

Gherkin Feature File — JetBrains Mono

```
When I sell 12 shares... proceeds are...
```

Java Model — JetBrains Mono

```
Portfolio.sell(...) -> SellResult.proceeds()
```

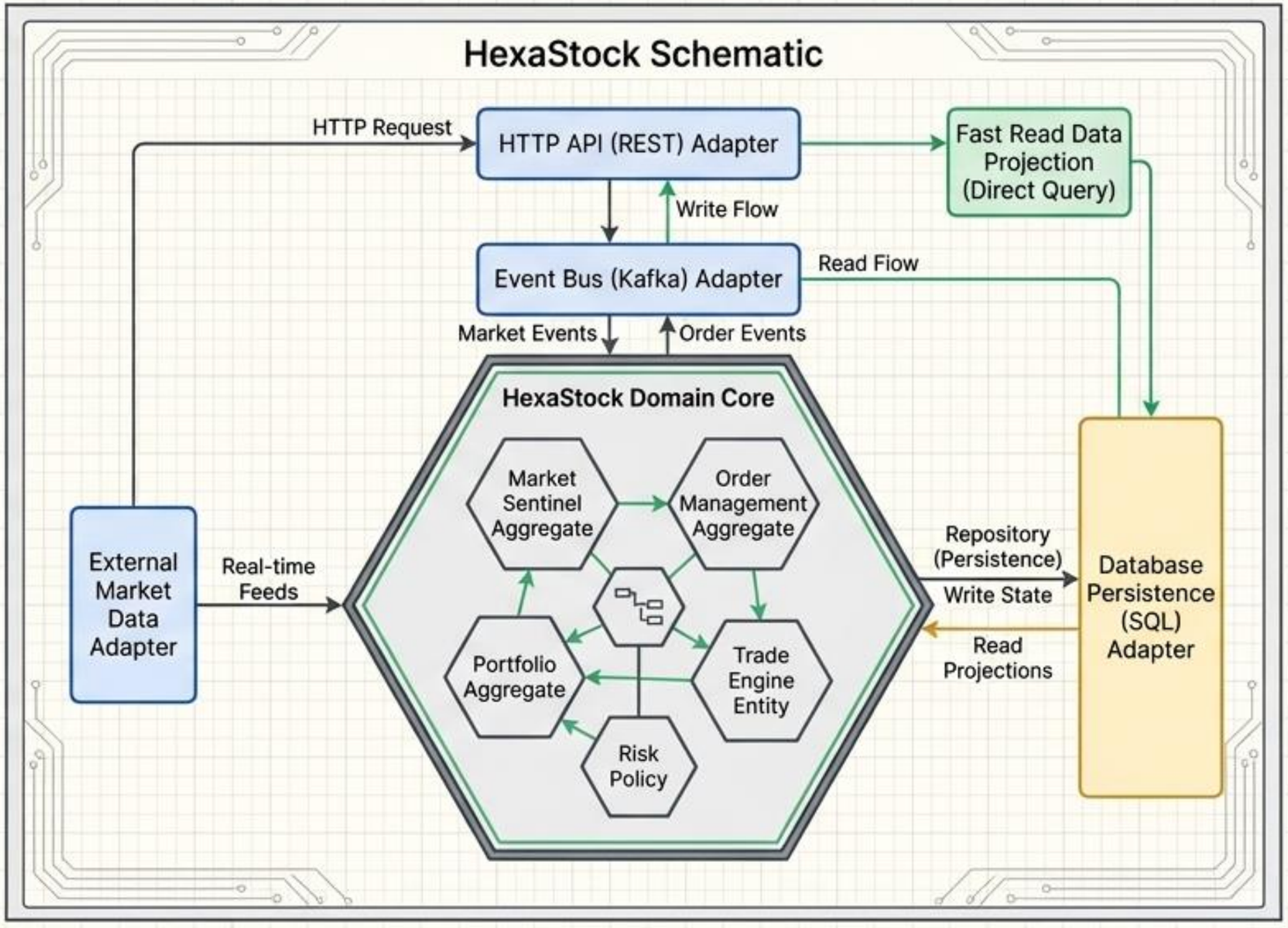
JUnit Result — JetBrains Mono

```
@SpecificationRef
✔ proceeds
```

NotebookLM

The Human Responsibility

- ✓ Hexagonal Architecture protects system boundaries.
- ✓ Domain-Driven Design captures the business truth.
- ✓ Rich models kill bugs at the source; Anaemic models breed them.
- ✓ AI will write your boilerplate, but YOU must define the boundaries.



NotebookLM

Continue the Journey

```
> git clone github.com/alfredorueda/HexaStock
```

Explore the Code

Examine the Java 21 & Spring Boot 3 open-source repository.

Run the Tests

Execute 170+ domain tests. Toggle between the Rich and Anaemic branches to watch failures happen.

Master the Architecture

Join AmigosCode Academy for comprehensive architectural teardowns.

https://amigoscode.com/

NotebookLM